

CAPSTONE: BOSS LEVEL

Project Panopticon: Intelligent Exam Proctoring

The Scenario

Welcome to the Machine Learning division of **EduGuard AI**, a leading online proctoring software company. With the massive shift to remote education, universities are using EduGuard to monitor students during final exams. Your software tracks their eye movements, background audio, and computer system logs to detect cheating.

However, the current system is a disaster. It is blindly flagging thousands of innocent students for simply sneezing, reading a question out loud, or looking at their keyboard. The universities are threatening to cancel their contracts.

Your mission is to build an intelligent, time-series classification pipeline that accurately detects genuine suspicious behavior while mathematically minimizing false accusations.

The Four Deadly Traps

You will face four real-world traps designed to test your ability to handle asynchronous data and ethical machine learning constraints.

1. **The Asynchronous Merge Trap (Pandas):** You have two datasets: `video_telemetry.csv` (recording eye gaze and head posture every 1 second) and `system_events.csv` (recording tab switches and copy/paste events *only when they happen*). You cannot do a standard merge. You must use Pandas `merge_asof` or time-based windowing to align asynchronous events to the nearest second.
2. **The “Connection Dropped” Trap (Data Cleaning):** Students have bad internet. The video feed drops out frequently, resulting in massive chunks of `NaN` values for their eye-tracking data. If you drop these rows, you lose the exam timeline. You must use Pandas interpolation or forward-filling to handle missing sensor data mathematically.
3. **The Micro-Movement Noise Trap (Feature Engineering):** Looking away from the screen for 1 second is not cheating. Looking away for 15 seconds is. You must engineer **Rolling Averages** (e.g., a 10-second rolling window) on the gaze and audio telemetry so your model doesn’t overreact to a student sneezing or stretching.
4. **The False Accusation Trap (ML & Business Logic):** A False Negative (missing a cheater) hurts the test’s integrity. But a False Positive (accusing an innocent student of cheating) destroys an academic career. You cannot use standard `.predict()`. You must use `.predict_proba()` and mathematically adjust the **Decision Threshold** of your Scikit-Learn model to prioritize *Precision* over raw accuracy, ensuring the system only flags a student if it is 90%+ certain.

Your Final Deliverable

You must submit **exactly one** artifact: **The Executive Jupyter Notebook**.

This single `.ipynb` file must run flawlessly from top to bottom. It must contain:

- All Pandas code used to mathematically handle dropped video frames and align the asynchronous time-series logs.
- Your complete feature engineering pipeline, proving you smoothed out the data using rolling-window logic.
- The training and execution of your Classification Model, properly weighted to handle the heavy class imbalance.
- A final evaluation segment featuring a Matplotlib Precision-Recall curve, along with your custom threshold implementation that protects innocent students.